FRONTEND DEVELOPER · WEB 3D & INTERACTION

오창민

건축을 전공한 프론트엔드 개발자입니다.

모형을 만지던 손으로, 지금은 웹에서 3D를 만지고 있습니다. 건축을 공부하며 모형과 3D 모델을 만들다가, 화면 위에서 직접 움직이는 공간이 궁금해 개발로 넘어왔습니다. SSAFY 14기에서 Vue와 React로 세 프로젝트의 화면과 3D 연출을 만들었고, 세 프로젝트 모두 상을 받았습니다. 지금은 LG전자 K-뉴딜 아카데미에서 AI로 업무 흐름을 자동화하는 AX 워크플로우를 배우고 있습니다.

EDUCATION	SSAFY 14기 수료 · 계명대학교 건축학 학사
EXPERIENCE	건축사사무소 인턴 (학부 4학년) — 모형 제작, 3D 모델링
NOW	LG전자 K-뉴딜 아카데미 1기 AX 워크플로우 트랙 (2026.08~) — 데이터 냉장고
AWARDS	SSAFY 공통 최우수상 · 특화 우수상 · 자율 우수상



OH CHANGMIN

dhckdals1104@naver.com

github.com/mm-sj

포트폴리오 웹사이트
mm-sj.github.io ↗

Skills

FRONTEND	React · Vue 3 · TypeScript · Zustand · Pinia
WEB 3D	Three.js · React Three Fiber
3D ASSET	Blender
DESIGN	AutoCAD · SketchUp · Adobe Photoshop
REALTIME	WebSocket
AUTOMATION	n8n
COLLAB	Git · Jira · Figma · Canva

Projects

01 **DocQ** — 공통 최우수상
PDF를 퀴즈로 만들어 함께 푸는 3D 보드게임

02 **Would You Draw** — 특화 우수상
그린 감정이 별이 되어 쌓이는 3D 우주 아카이브

03 **Jabis** — 자율 우수상
양팔 데스크 로봇의 실시간 디지털 트윈 대시보드

04 **데이터 냉장고** — LG AX 트랙
저장한 이유가 다시 유효해질 때 자료를 꺼내주는 AX 워크플로우

SHEET	PERIOD	TEAM	ROLE	STACK	AWARD
01 / 04	2026.01.06 – 02.13	6명 (FE 3)	FE · UX 방향 · 3D 에셋	Vue 3 Three.js	공통 최우수상



01 — DocQ

DocQ

PDF 문서를 AI가 퀴즈로 바꾸고, 최대 4명이 3D 보드게임으로 푸는 실시간 멀티플레이 서비스

'다 읽었다'와 '다 안다'는 다릅니다. 읽은 문서를 제대로 이해했는지 혼자 검증하는 지루한 과정을, 친구와 겨루는 게임으로 바꾸는 것이 목표였습니다. 저는 이 게임이 어떤 모습이고 어떻게 느껴질지를 정하는 역할을 맡았습니다.



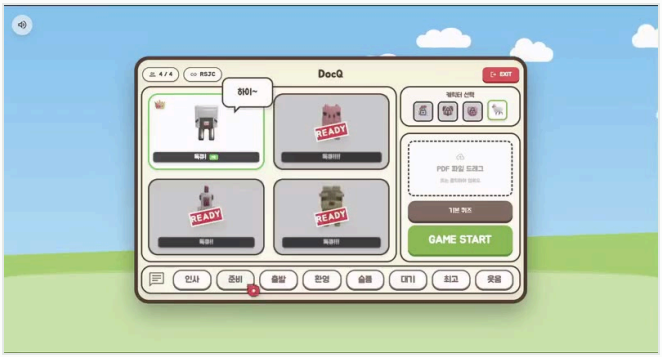
메인 화면 — 3D 캐러셀 메뉴와 GLB 캐릭터 배경



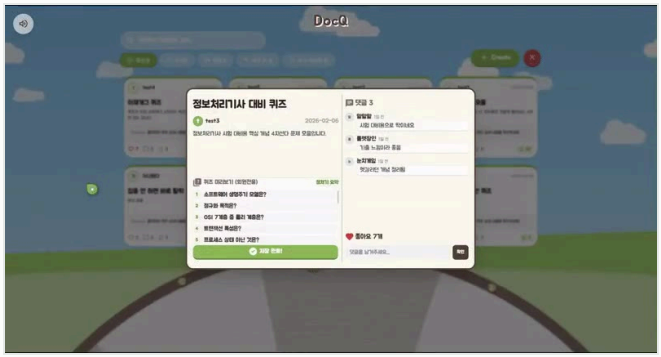
Blender로 제작한 보드 맵



이동 칸 수를 정하는 3D 룰렛



대기실 — 캐릭터 선택, PDF 업로드, 채팅 매크로



퀴즈 공유 커뮤니티

제가 한 일

- **비주얼·UX 방향 수립.** 나무 패널, 파스텔 하늘, 둥근 버튼으로 이어지는 동화 같은 톤을 잡고 화면 전반에 적용했습니다.
- **3D 에셋.** 가져온 에셋을 Blender로 수정해 보드 맵을 만들고, 캐릭터와 룰렛의 애니메이션 좌표를 다시 설정했습니다.
- **메인 화면.** CSS 3D 캐러셀 메뉴와, 캐릭터 4종이 동시에 움직이는 Three.js 배경(AnimationMixer)을 구현했습니다.
- **게임 밖 화면 전반.** 로비, 대기실, 커뮤니티(게시글·댓글 삭제와 확인 모달), 미니게임 UI, 커스텀 커서, BGM, 팀 소개 모달을 만들었습니다.

Vue 3 · TypeScript · Pinia · TailwindCSS · Three.js · GSAP · Blender

트러블슈팅

룰렛이 제자리에서 돌지 않고 궤도를 그리며 돌던 문제

문제	수정한 룰렛 GLB를 웹에 올리자, 원판이 자기 중심이 아니라 엉뚱한 지점을 축으로 크게 원을 그리며 돌았습니다. Blender를 처음 다뤄 본 상태라 원인을 찾는 데 오래 걸렸습니다.
원인	Blender 오브젝트의 원점(Origin)이 원판 중심이 아닌 다른 위치에 있었습니다. Three.js는 오브젝트를 원점 기준으로 회전시키기 때문에, 원점이 어긋나면 회전축도 함께 어긋납니다.
해결	Blender에서 원판의 원점을 회전 중심으로 다시 지정하고 변환(Transform)을 적용한 뒤 다시 내보냈습니다. FE에서 모델의 바운딩 박스 중심으로 보정하는 로직과 맞춰 룰렛이 제자리에서 회전하도록 했습니다.
배운 점	3D 에셋은 눈에 보이는 모양보다 원점, 축, 스케일 같은 보이지 않는 데이터가 웹에서의 동작을 결정한다는 것을 알게 됐습니다.

공통 최우수상
SSAFY 14기 공통 프로젝트

92%
팀이 받은 유저 피드백 24건 중 22건 반영

GitHub 저장소 ↗

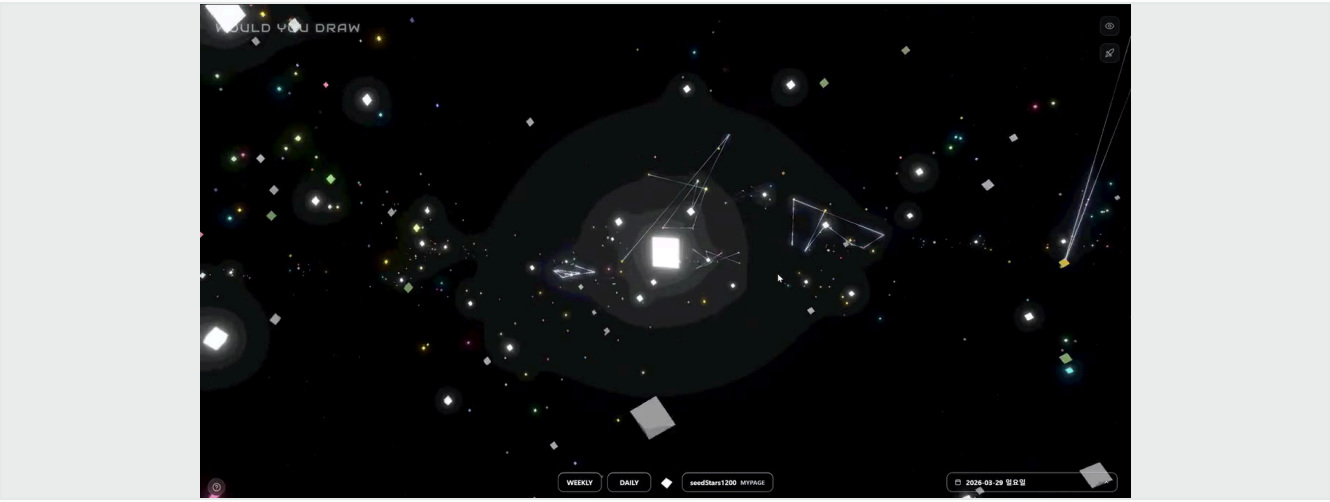
SHEET	PERIOD	TEAM	ROLE	STACK	AWARD
02 / 04	2026.03.03 – 03.29	5명 (FE 2)	FE 팀장	React R3F	특화 우수상

02 — Would You Draw

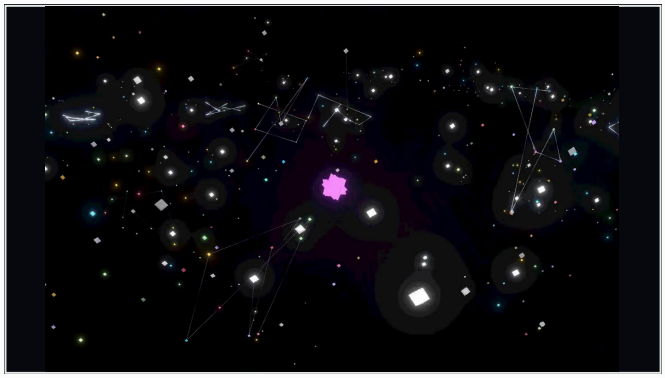
Would You Draw

감정을 그림으로 그리면 AI가 해석하고, 그 기록이 별에서 별자리(7일), 은하(30일)로 쌓이는 감정 드로잉 PWA

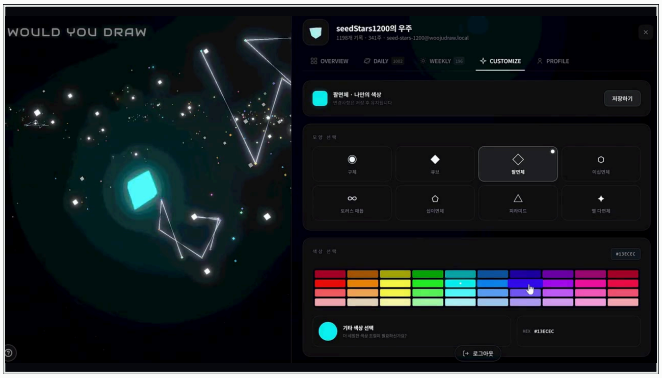
감정을 점수로 매기는 대신, 시간이 지날수록 넓어지는 우주로 보여주고 싶었습니다. FE 팀장으로 우주 씬의 인터랙션과 드로잉 콘텐츠를 맡았습니다.



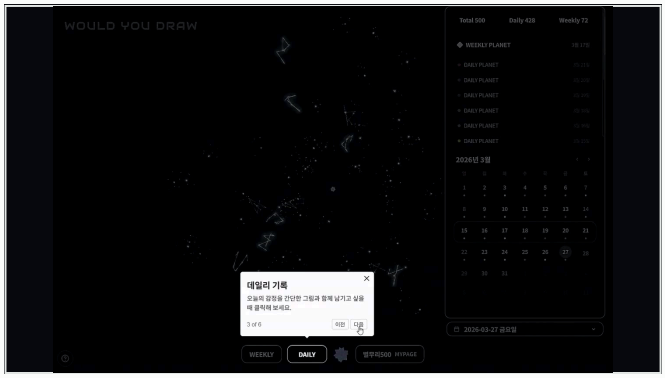
감정 기록이 모여 만든 우주 — 은하에서 개별 별까지 카메라가 이어서 이동



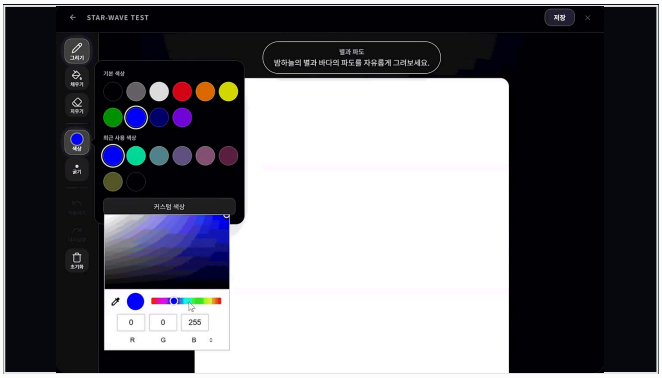
새 별과 별자리



중심별 형태·색 커스텀



피드백 반영 튜토리얼



드로잉 캔버스 — 커스텀 컬러 팔레트

제가 한 일

- **우주 씬 인터랙션.** 팀원이 만든 R3F 우주 씬 위에서 별자리 연결 로직, 새 별 탄생 연출, 시네마틱 모드, 패널이 열릴 때 카메라 구도를 옮기는 레이아웃을 맡았습니다.
- **드로잉 캔버스.** 캔버스를 공통 컴포넌트로 분리하고, HTP 단계별 드로잉, 데일리 자유 드로잉, 명화 색칠, 커스텀 컬러 팔레트를 구현했습니다.
- **분석 결과와 커스텀.** HTP 분석 결과 UI와, 8가지 형태와 팔레트로 꾸미는 마이페이지 중심별 커스텀을 만들었습니다.
- **피드백 대응.** 유저 피드백 37건에서 반복된 '직관성·가이드 부족'에 맞춰 튜토리얼과 전역 알림 모달을 넣었습니다.

React · TypeScript · Zustand · React Three Fiber · drei · postprocessing · Framer Motion · Canvas API

트러블슈팅

저장해도 별이 바로 뜨지 않던 문제 — 낙관적 업데이트

문제

그림을 저장한 뒤 AI 분석이 RabbitMQ로 비동기 처리되어, 분석이 끝나기 전까지는 별이 생기지 않았습니다. 저장 직후 우주에 아무 변화가 없어 사용자는 저장이 실패한 것으로 느낄 수 있었습니다.

해결

저장 응답을 받는 즉시 임시 별을 만들고 탄생 애니메이션을 재생했습니다. 이후 `2 → 5 → 10 → 15 → 20초` 간격으로 별 목록을 다시 조회하고, 서버에서 실제 별이 오면 `targetId`로 임시 별을 대체했습니다. 폴링 중에는 새 폴링을 막아 요청이 겹치지 않게 했습니다.

배운 점

비동기 처리가 끝날 때까지 화면이 기다리게 하지 않고, 결과를 먼저 보여준 뒤 나중에 맞추는 방식으로 체감 속도를 바꿀 수 있다는 것을 배웠습니다.

특정 색의 별만 빛나지 않던 문제

사용자가 고른 감정 색으로 별을 그리는데, 일부 색의 별은 Bloom 효과가 나타나지 않았습니다. 같은 발광 강도라도 색마다 휘도가 달라, 휘도가 낮은 색은 Bloom 임계값(`1.1`)을 넘지 못했기 때문입니다. 임계값을 `0.5`로 낮추고 반짝임 범위를 넓혀 모든 색이 고르게 빛나도록 했습니다.

특화 우수상
SSAFY 14기 특화 프로젝트

37건
유저 피드백 수집 → 튜토리얼·캔버스·별 렌더링 개선

GitHub 저장소 ↗

SHEET	PERIOD	TEAM	ROLE	STACK	AWARD
03 / 04	2026.04.08 – 05.21	6명 (FE 1)	FE 전담	React R3F URDF	자율 우수상

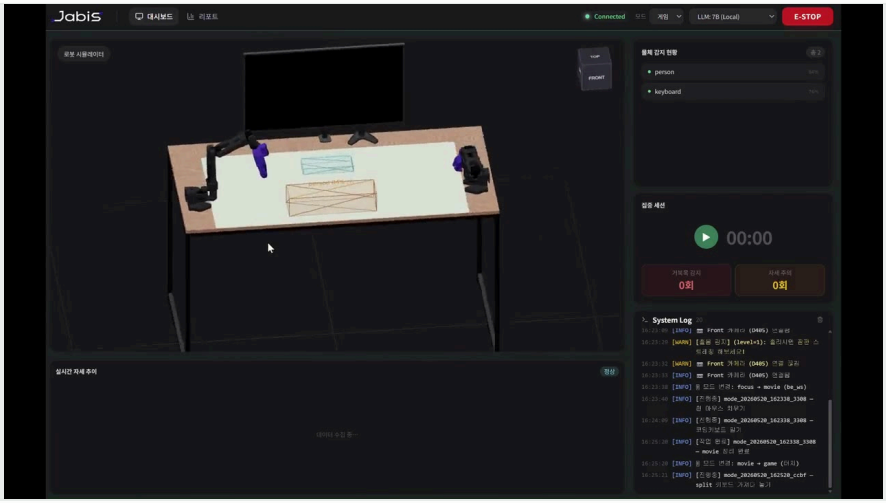


03 — Jabis

Jabis

음성 명령을 듣고, 책상을 보고, 두 팔로 물건을 집는 Physical AI 데스크 로봇의 웹 대시보드와 디지털 트윈

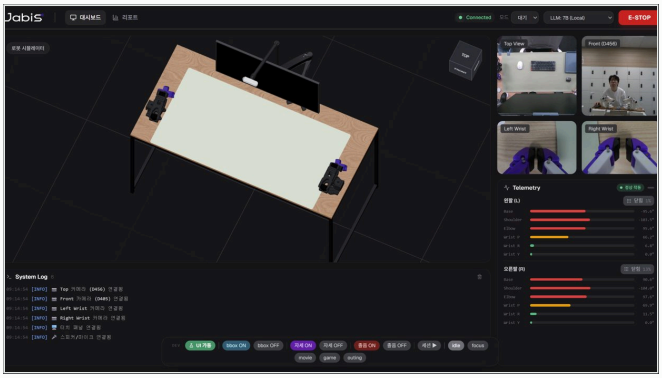
로봇이 지금 무엇을 보고 어떻게 움직이는지 사람이 한눈에 알 수 있어야 했습니다. 팀의 유일한 FE로서, 로봇의 관절·인식·자세 데이터를 실시간으로 받아 3D로 보여주는 대시보드와 디지털 트윈을 맡았습니다.



실측 책상(1200 × 600 × 700 mm) 위의 양팔 로봇 디지털 트윈과 인식된 물체의 3D 박스



실제 양팔 로봇 — 대시보드의 3D가 따라 움직이는 원본



카메라 4대 영상과 양팔 관절 텔레메트리 패널

제가 한 일

- **디지털 트윈.** SO-ARM101 URDF를 불러와 양팔 로봇을 실측 책상 GLB 위에 배치하고, WebSocket으로 받은 관절 각도를 실시간으로 반영했습니다.
- **인식 결과 시각화.** YOLO 인식 결과를 3D 박스로 렌더링하고, MJPEG 카메라 스트림과 시스템 로그, E-Stop을 연결했습니다.
- **모드별 대시보드.** 집중·게임·영화·외출 모드마다 레이아웃을 바꾸고, 자세 모니터링 차트와 리포트, 졸음 감지 카드를 만들었습니다.
- **FE 기반 설계.** FE 폴더 구조, FE API 명세, Zustand 스토어와 타입 정의를 세팅했습니다.

React · TypeScript · Vite · React Three Fiber · urdf-loader · Zustand · WebSocket · Recharts · TailwindCSS · shadcn/ui

트러블슈팅

실시간 인식 박스가 깜빡이던 문제 — 렌더링을 React 밖으로

문제

탑뷰 카메라의 YOLO 인식 결과(약 5Hz)가 WebSocket으로 들어올 때마다 3D 박스 목록 전체가 다시 렌더링되었습니다. 여기에 YOLO가 한두 프레임씩 물체를 놓치면 박스가 사라졌다 다시 생기면서 화면이 깜빡였습니다.

해결

Zustand의 `subscribe`로 인식 결과를 React 렌더링 밖에서 받아 `ref`의 Map에 저장했습니다. 박스의 위치와 크기는 `useFrame` 안에서 lerp로 직접 움직이고, React 리렌더는 새 물체가 생기거나 오래 보이지 않은 물체를 지울 때만 일어나게 했습니다. 마지막 감지 후 박스를 유지하는 시간은 처음 800ms로 두었다가, 실제 로봇에서 YOLO가 놓치는 구간을 보고 2초(5Hz 기준 10프레임)로 늘렸습니다.

배운 점

고빈도 실시간 데이터는 React 상태로 흘려보내지 않고, 화면에 필요한 변화만 React에 알리는 편이 낫다는 것을 배웠습니다.

로봇 움직임을 부드럽게 만든 작업

- 20Hz로 들어오는 관절 데이터를 60fps 렌더 사이에서 lerp로 보간했습니다.
- 그리퍼 모터 각도를 링키지 관절 각도로 바꾸는 룩업 테이블을 선형 보간했습니다.
- URDF의 Z-up 좌표계를 Three.js의 Y-up에 맞게 변환했습니다.
- 탑뷰 카메라 bbox의 픽셀 좌표를 책상 평면의 미터 좌표로 바꾸고, 물체 종류별 높이를 추정해 3D 박스를 세웠습니다.
- 조인트 참조를 한 번만 찾아 캐싱하고, 움직이지 않는 책상 메시는 `matrixAutoUpdate`를 꺼 매 프레임 계산을 줄였습니다.

자율 우수상
SSAFY 14기 자율 프로젝트

FE 1인
팀 내 유일한 FE로 대시보드·디지털 트윈 담당

홍보 영상 보기 ↗ · GitHub 저장소 ↗

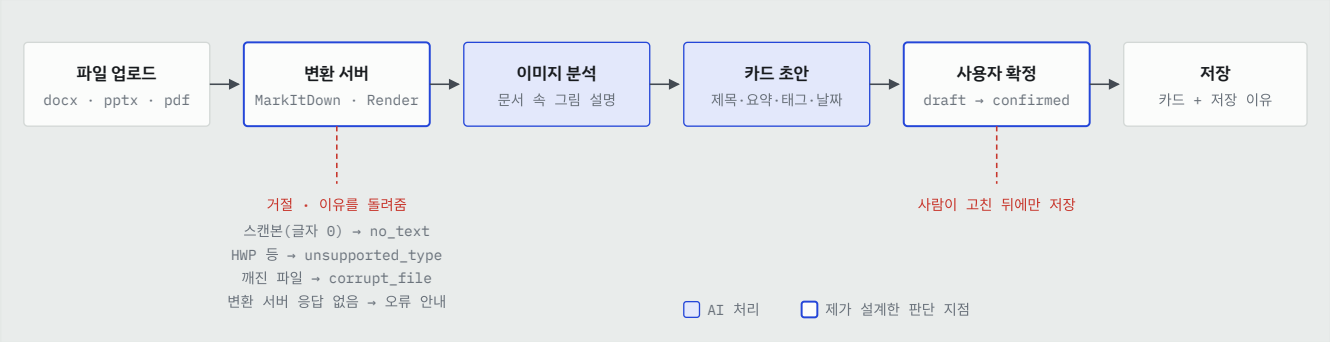
SHEET	PERIOD	TEAM	ROLE	STACK	TRACK
04 / 04	2026.10.19 – 11.04	5명	자료 입고 워크플로우 · 웹 화면	n8n FastAPI MarkItDown Three.js	LG전자 AX

04 — 데이터 냉장고

데이터 냉장고

자료를 저장할 때 이유를 남기면, 그 이유가 다시 유효해지는 업무 상황에서 AI가 자료를 꺼내주는 사내 지식 재활용 서비스

회사 자료는 쌓여도 '왜 저장했는지'가 남지 않아 다시 쓰이지 않습니다. LG전자 K-뉴딜 아카데미 AX 워크플로우 트랙의 팀 프로젝트로, 저장 이유와 활용 조건을 자료와 함께 남기는 구조를 n8n으로 만들었습니다. 저는 자료가 들어오는 입구, 즉 어떤 파일이든 AI가 읽을 수 있는 카드로 바꾸는 흐름을 맡았습니다.



자료 입고 흐름 — AI가 만든 초안은 사람이 확인해야 저장됩니다

제가 한 일

- **입고 워크플로우 설계·구현.** 파일 → 마크다운 변환 → 이미지 분석 → AI 카드 초안 → 사용자 확정 → 저장으로 이어지는 n8n 흐름을 만들었습니다. 팀원 버전과 비교를 거쳐 팀의 기본 구조로 채택됐습니다.
- **변환 서버 배포.** 형식이 제각각인 문서를 AI가 읽을 텍스트로 바꾸는 MarkItDown 서버(FastAPI)를 Docker로 Render에 배포하고, API 키 인증을 붙여 n8n과 연결했습니다. n8n 안에서 변환하는 방법(Execute Command, Code 노드)도 검토했지만 환경 제약이 커서 별도 서버로 정했습니다.
- **사람 검토 단계.** AI가 만든 요약과 태그를 바로 저장하지 않고, 사용자가 확인·수정한 뒤 확정하도록 초안(draft)과 확정(confirmed) 상태를 나눴습니다.
- **웹 화면.** 양문형 냉장고를 열어 '넣기'와 '꺼내기'를 고르는 화면을 와이어프레임하고, three.js와 GSAP로 3D 프로토타입을 만들었습니다.

n8n · Python · FastAPI · MarkItDown · Docker · Render · Three.js · GSAP

트러블슈팅

깨진 파일이 '변환 성공'으로 통과하던 문제

문제

실패 경로를 테스트하다가, 확장자만 docx이고 내용은 깨진 파일을 올렸는데 서버가 성공 응답을 돌려줬습니다. 의미 없는 글자가 그대로 AI 카드 초안 단계로 넘어갈 수 있는 상태였습니다.

원인

MarkItDown은 형식을 읽지 못하면 오류를 내는 대신 파일을 일반 텍스트로 읽고 넘어갑니다. 실패가 성공처럼 보였습니다.

해결

변환 전에 파일 앞부분의 시그니처를 확장자와 대조해 맞지 않으면 `corrupt_file`로 거절했습니다. 같은 기준으로 글자 수가 0인 스캔본은 `no_text`, 지원하지 않는 형식은 `unsupported_type`으로 나눠, n8n이 이유별로 사용자에게 안내하게 했습니다.

배운 점

AI 자동화에서 더 위험한 것은 멈추는 실패가 아니라 조용히 통과하는 실패라는 것을 알게 됐습니다. AI 앞단에서 입력을 먼저 걸러야 결과를 믿을 수 있습니다.

팀 기본안 채택
제가 만든 입고 워크플로우

4가지 실패 응답
스캔본 · 미지원 형식 · 깨진 파일 · 서버 다운

소개 영상 보기 ↗